Case Study: Strangler Pattern at Blackboard Learn

Jose Velazquez Saenz

Bellevue University

CSD380: DevOps

Professor Sue Sampson

July 2, 2026

Case Study: Strangler Pattern at Blackboard Learn

The case study "Strangler Fig Pattern on Blackboard Learn" explains how Blackboard Inc. addressed the challenges of a large, legacy software system. Blackboard Learn is an educational technology platform, but by 2011, the company was facing problems stemming from an outdated, complex codebase. The system had been around since 1997 and was built on a legacy J2EE codebase. David Ashman, the chief architect, noted that there were still fragments of Perl code inside the system. This shows how software can become harder to manage over time when older technologies remain mixed into newer development.

One of the author's main points is that Blackboard's development process had become slow, complicated, and risky. By 2010, the company was experiencing long build, integration, and testing cycles. A full feedback cycle could take between twenty-four and thirty-six hours. This meant developers had to wait a long time to find out whether their code worked correctly. Long feedback times can reduce productivity because developers cannot quickly identify and fix problems. As the product continued to grow, the development process became more difficult, resulting in worse outcomes for customers. The graphs in the case study help show the problem. The first graph showed that the number of lines of code in the main Blackboard Learn repository continued to increase. However, the number of code commits started to decrease. This was a warning sign because it showed that the system was getting larger while developers were making fewer changes. The author explains that this meant it was becoming harder and riskier to introduce new code changes. Ashman realized that if the company did not do something, the problems would continue to get worse.

To solve this issue, Blackboard began using the Strangler Fig Pattern in 2012. This pattern allows a company to modernize a legacy system gradually rather than replacing it all at once. Rather than rewriting the whole application, developers slowly move parts of the system into new modules or services. Over time, the new system begins to replace parts of the old system. This approach is less risky

because the company can continue supporting users while improving the software. At Blackboard, this strategy was implemented through a feature called Building Blocks. Building Blocks allowed developers to create separate modules that were decoupled from the main monolithic codebase. These modules are still connected to the original system through fixed APIs. This gave developers more independence because they could work on separate parts of the application without constantly coordinating with every other team. It also made the system safer because problems in one module were less likely to break the entire platform.

The second graph showed the results after Building Blocks were introduced. The main code repository began to shrink as developers moved code into the Building Blocks repositories. At the same time, activity in the Building Blocks repositories increased. This showed that developers could work more productively in smaller, more manageable areas of the system. Breaking the system into smaller parts made development easier, safer, and more flexible.

The overall lesson from this case study is that a single large rewrite should not always replace legacy systems. A full rewrite may seem like a clean solution, but it can be expensive, risky, and disruptive. Blackboard's use of the Strangler Fig Pattern shows that gradual modernization can be a better option. By moving functionality into Building Blocks, Blackboard improved the system while keeping the existing product available to customers. Another lesson is that technical debt often appears through warning signs such as long build times, slow testing, fewer commits, and developer frustration. These problems are not just technical issues; they affect the business by slowing the delivery of new features and fixes. Blackboard's experience shows that organizations need to pay attention to these signs before the system becomes too difficult to maintain.

The case study shows that Blackboard Learn improved its legacy system by gradually breaking apart its monolithic codebase. The Strangler Fig Pattern helped the company reduce risk, improve developer productivity, and make future changes easier. The most important lesson is that modernizing

software does not have to happen all at once. A careful, step-by-step approach can make large systems easier to maintain while protecting customers from major disruptions.

References

Kim, G., Humble, J., Debois, P., Willis, J., Forsgren, N., & Allspaw, J. (2021). *The devops handbook: How to create world-class agility, reliability, & Security in Technology Organizations*. IT Revolution Press.